

Python Desktop Poker Development

CustomTkinterを用いたデスクトップアプリ開発

中間制作報告：プログラミング課題

制作物の概要

カリキュラムで習得した知識の集大成としての実装

コンセプトと要件定義

開発コンセプト

「Pythonの基礎知識を活用し、視覚的に楽しめる実用的なアプリケーションの構築」

オブジェクト指向(OOP)を基盤に、拡張性の高いコード設計を目指しました。

主要な要件

- ✓ 直感的なGUI
- ✓ 高度な役判定アルゴリズムの実装
- ✓ Excelによる戦績データの保存・管理
- ✓ ダブルアップ等のゲーム性向上機能

システム構成とクラス設計

クラスベースの設計

責務を明確に分けた4つの主要クラス：

Card / Deck: カード情報と山札の管理

Player: 手札とチップ、役判定のロジック

PokerApp: GUI制御と全体のゲーム進行

各クラスが独立しているため、ロジックの修正がGUIに影響を与えない疎結合な設計です。

```
class Card:
    def __init__(self, suit, value): ...

class Deck:
    def shuffle(self): ...
    def draw(self): ...

class Player:
    def judge_hand(self): ...

class PokerApp(ctk.CTk): ...
```


オブジェクト指向設計の実践



カプセル化

Cardクラスは属性を保持し、Deckクラスは「シャッフル」等の振る舞いをメソッド内に隠蔽しています。



責務の分離

Playerがデータ判定、PokerAppがGUI制御を担当。UIとロジックを分離し、可読性を高めています。



状態の管理

インスタンス変数を活用し、チップ残高や手札の状態をラウンドを跨いで正確に保持しています。

役判定アルゴリズムの解説

```
# 役判定のコアロジック
def judge_hand(self):
    nums = sorted([c.value for c in hand], rev=True)
    counts = Counter(nums)
    pattern = sorted(counts.values())

    is_flush = len(set(c.suit for c in hand)) == 1
    is_st = all(nums[i]-nums[i+1]==1 for i in range(4))

    if is_flush and is_st: return "St-Flush"
    elif pattern == [1, 4]: return "4-Cards"
    elif pattern == [2, 3]: return "Full-House"
```

judge_hand() のロジック

- </> リスト内包表記とCounter: 手札の数値を集計し、役のパターンを特定。
- ↑? 役の優先度判定: 最上位の役から順に条件分岐 (if-elif) で特定。
- ✂ 例外処理: A-5ストレート等の固有規則も論理式でカバー。

GUI : CustomTkinter



Dark Theme

CustomTkinter独自のダークモードを採用し、モダンで高級感のあるゲーム画面を演出。



動的レイアウト

グリッドシステムを活用し、ウィンドウサイズに応じた柔軟な配置を実現しています。



視覚的な一貫性

カラーパレットを統一し、ボタンやラベルのデザインに一貫性を持たせました。

ユーザー体験と視覚フィードバック



カード交換

チェックボックスによる直感的な操作とUI上の即時更新を連動。

ユーザーがどのカードを残し、どのカードを捨てるかを一目で認識できるよう、選択状態をリアルタイムで明示する視覚効果を実装しました。



結果通知

勝敗に応じた分かりやすい配色でユーザーへ現在のゲーム状況をダイレクトに伝達。

勝利時は明るいトーン、敗北時は落ち着いたトーンを使い、煩わしいポップアップダイアログを挟まずにテンポよく結果を認知させる設計です。



ベット管理

不正な入力を完全に防ぐためのバリデーション（入力検証）付き入力制御を導入。

所持チップを超える過度なベット額や、不適切な文字入力をプログラム側で弾き、システムエラーを未然に防ぐ快適なUX（ユーザー体験）を実現。

ダブルアップ機能の設計

別ウィンドウによるミニゲーム

ゲーム性を高める追加機能：

基準カードに対する大小予想ロジック。

的中による配当2倍、または没収の条件分岐。

grab_set() によるモーダルウィンドウ制御。

x2

High or Low Logic

データ永続化：Excel連携

項目	保存されるデータ
日時	タイムスタンプ
ベット/役	各ラウンドの詳細
勝敗	対戦結果
所持チップ	残高の推移記録

poker_history.xlsx			
A	B	C	D
Timestamp	Bet	Role	Chips
2026-06-19	100	ツーペア	1200
2026-06-19	200	フルハウス	2800
2026-06-19	150	役なし	2650

開発における課題と試行錯誤



設計方針の模索

機能の着地点を定めないまま開始したため、発展性に課題を感じる場面もありました。



アーカイブの活用

多角的に検討するため、archiveフォルダを活用し論理構造の変遷を管理しました。



段階的進化

関数ベースからOOPへの移行を通じ、コードを洗練させる重要性を学びました。

今後の改善案とさらなる発展



テキサスホールデム化

コミュニティカードを使用する形式へ換装し、戦略性を高める改修を検討しています。



オンライン対戦

ネットワーク通信を実装し、他プレイヤーとのリアルタイム対戦機能を目指します。



CPU思考の強化

ディーラー側に「ブラフ」等の要素を加え、より高度な対戦相手を実装します。

カリキュラム学習の成果

変数・制御構文から外部ライブラリ活用まで
「実力として定着した知識」の証明

Thank You

ご清聴ありがとうございました。

使用素材・技術一覧

- ✓ フォント: Google Fonts (Noto Sans JP, Roboto) - SIL OFL
- ✓ アイコン: FontAwesome Free - SIL OFL / MIT License
- ✓ 画像: CSS描画およびコードビューを使用（著作権画像不使用）
- ✓ ライブラリ: openpyxl, CustomTkinter (MIT License)